

ECE 382N-Sec (FA26):

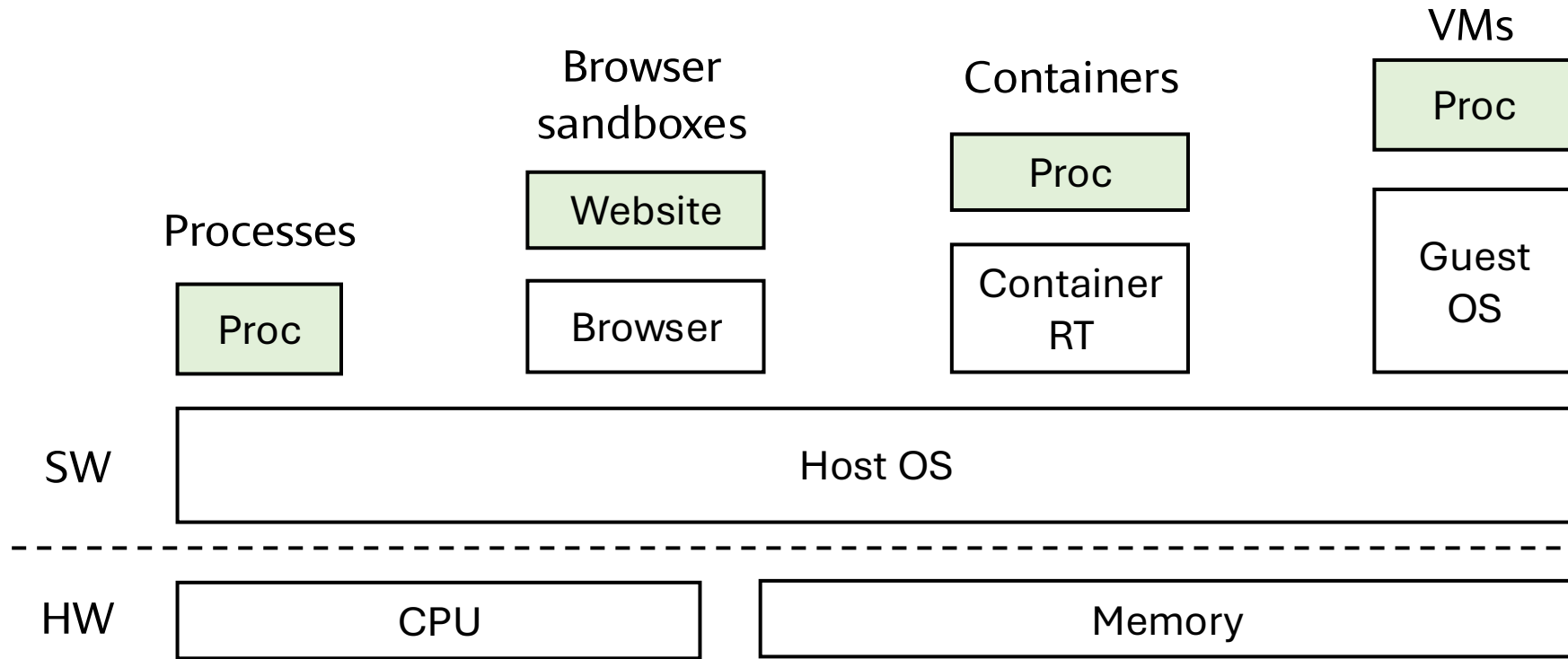
L8: Trusted Execution Environments (TEEs)

Neil Zhao

neil.zhao@utexas.edu

Different Isolation Techniques

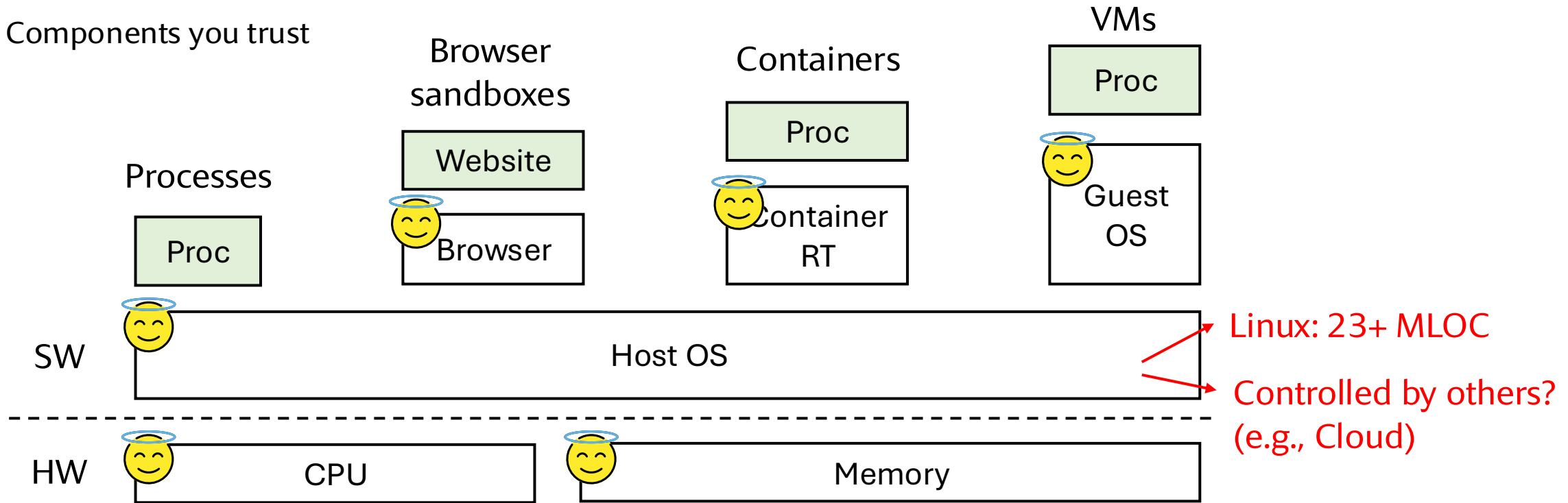
 Your program



Trusted Computing Base (TCB)

 Your program

 Components you trust

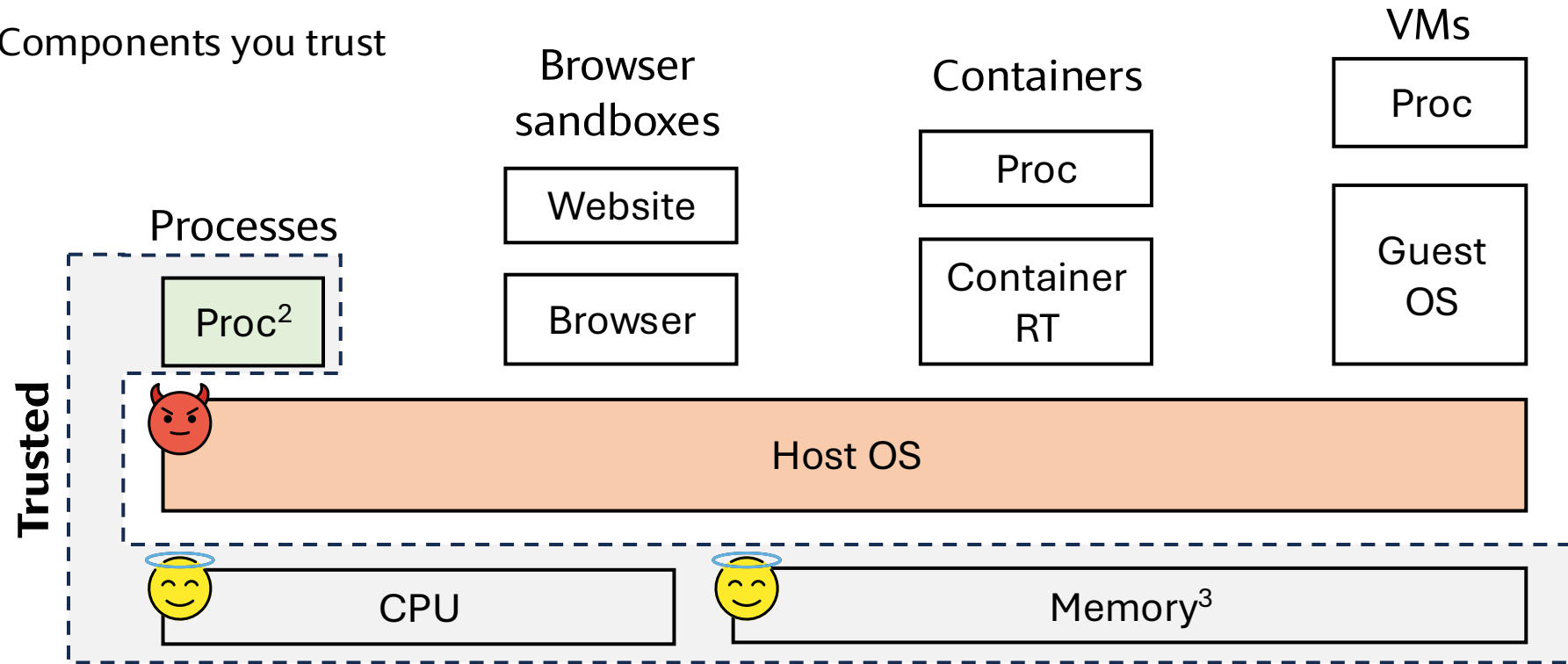


The only components that can betray us, are the ones we trust

Trusted-Execution Environments (TEE)¹

 Your program

 Components you trust



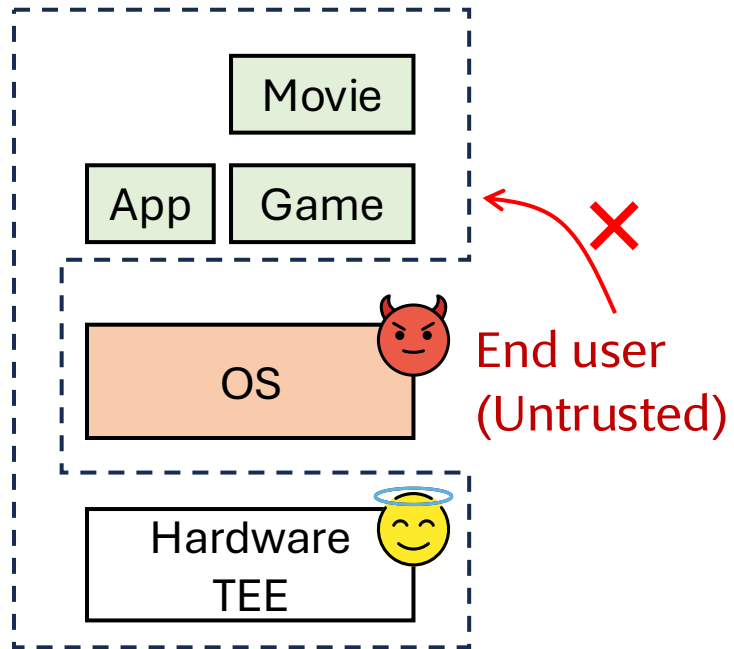
¹TEE is a somewhat overloaded term. We focus on hardware-based TEEs

²The process may be divided into trusted and untrusted parts

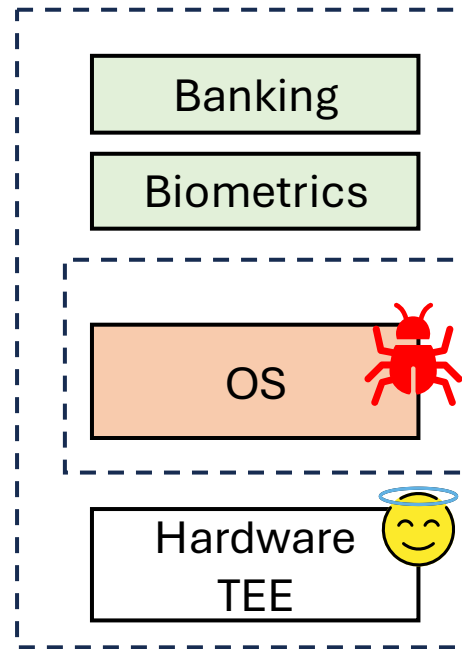
³Depending on the memory type and threat model, it may or may not be trusted

TEE Use Cases

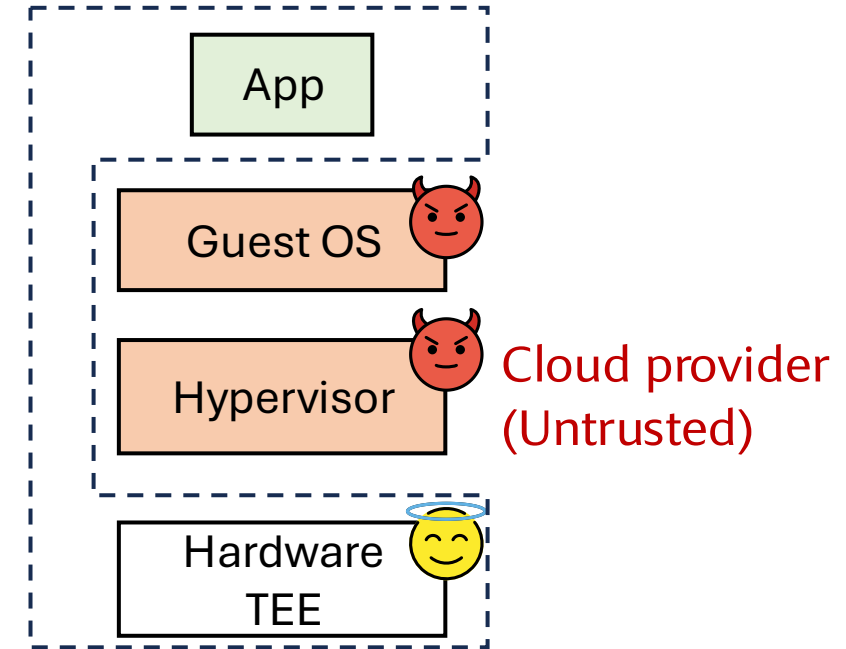
Copyright Protection



Minimizing TCB



Outsourcing Computation



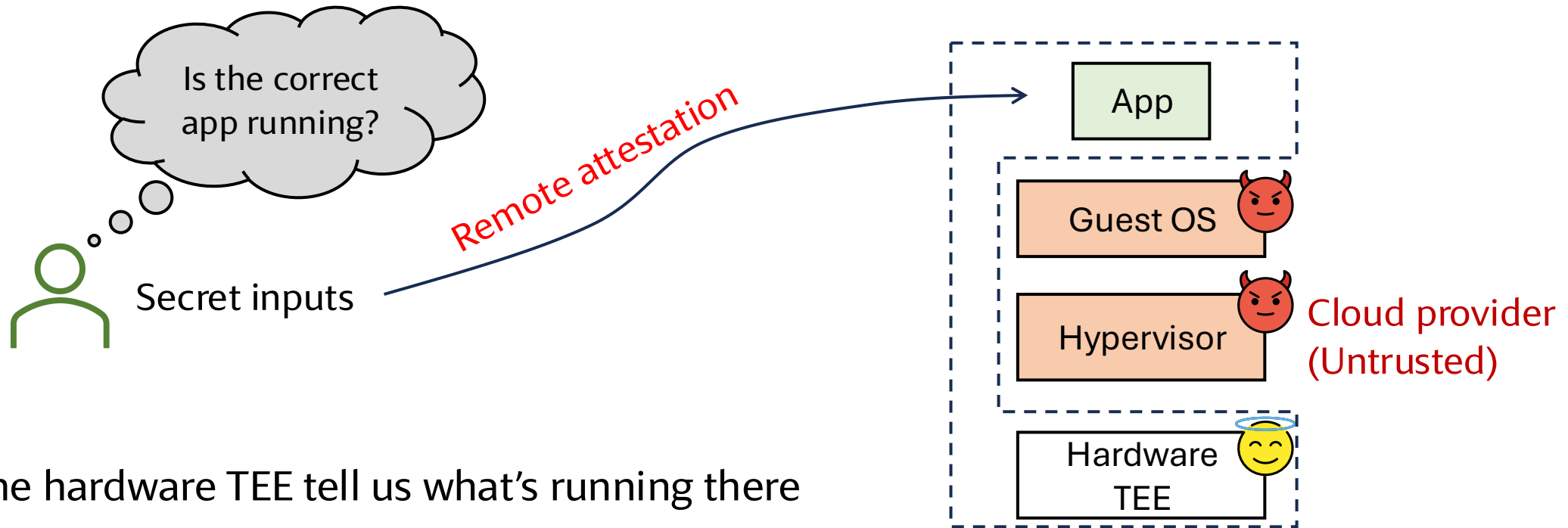
(Common*) Security Goals of TEEs

		Example Attacks	
		Software Attack	Physical Attack
✓	Confidentiality	Attacker cannot directly access my private program states (Side channel? Spectre?)	OS reads my pages Bus snooping
✓	Integrity	Attacker cannot tamper with my program states (Freshness: attacker cannot replay an older state)	OS writes my pages ? Bus spoofing
✗	Availability	Attacker refuses to execute or give enough resources to my program	OS allocates no CPU time Pull the plug

Note: Availability of the host is protected from my program in TEE---the OS can always terminate my program without my cooperation

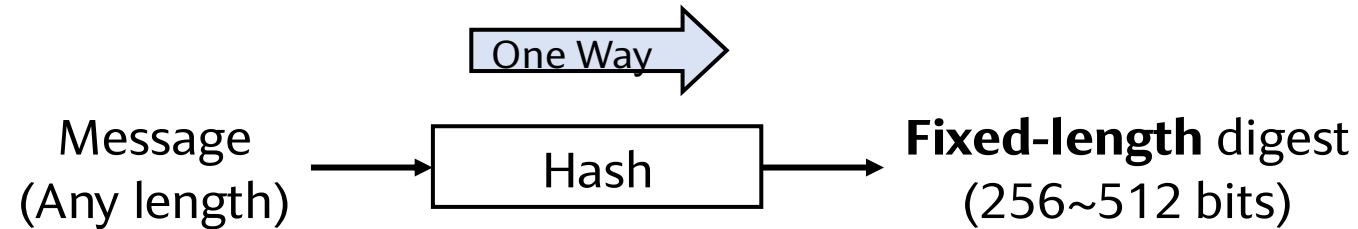
*Many variants exist

Establishing Trust?



Idea: Let the hardware TEE tell us what's running there

Hammer 1: Cryptographic Hash Functions



Avalanche effect (example uses SHA-256):

"44 students are registered for ECE-382N" → "f1246ddddd902aa8de27ddce24bd24031..."

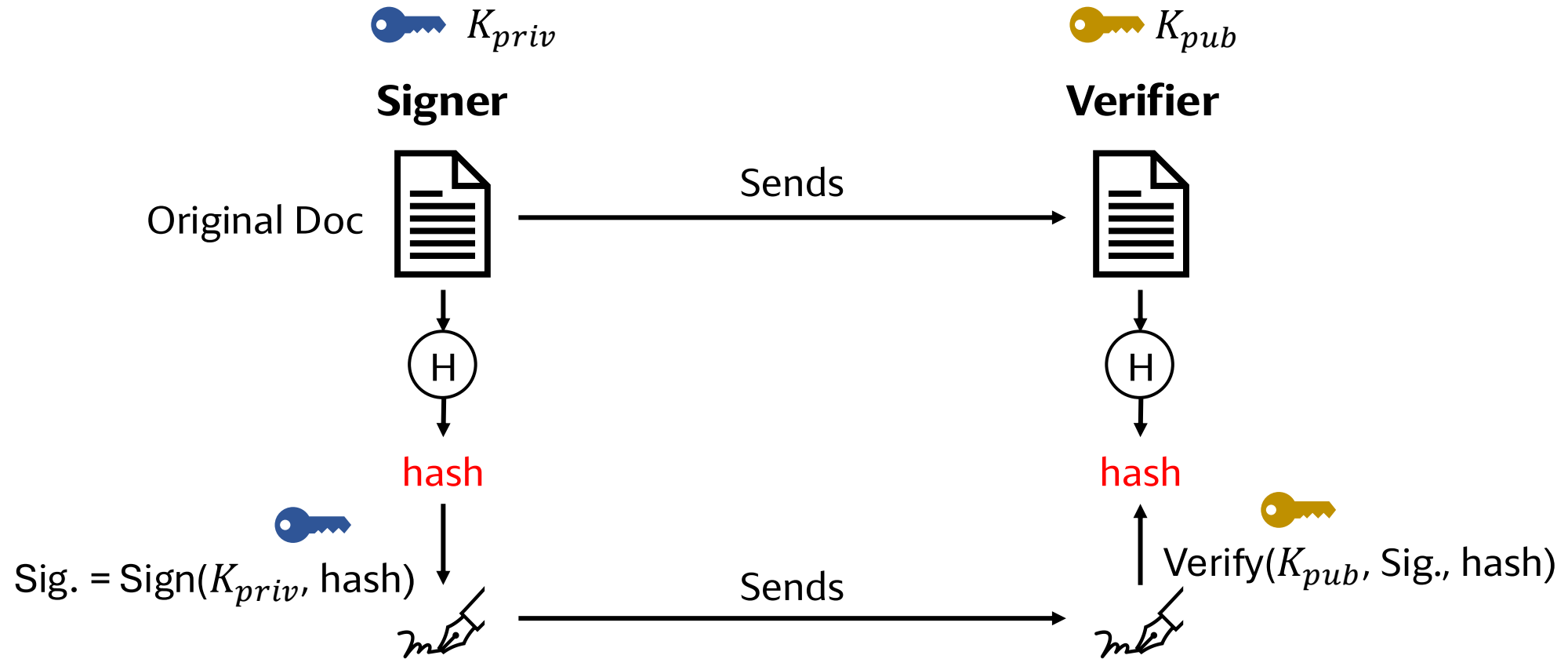
"43 students are registered for ECE-382N" → "4bb1db3619a8442661fc9107b1767483..."

Security properties:

- **Preimage resistance:** For any random hash value h , it's practically impossible to find a message M such that $\text{Hash}(M) = h$ in practice
- **Collision resistance:** Despite the inevitability of collision, it's practically impossible find two distinct messages that hash to the same value

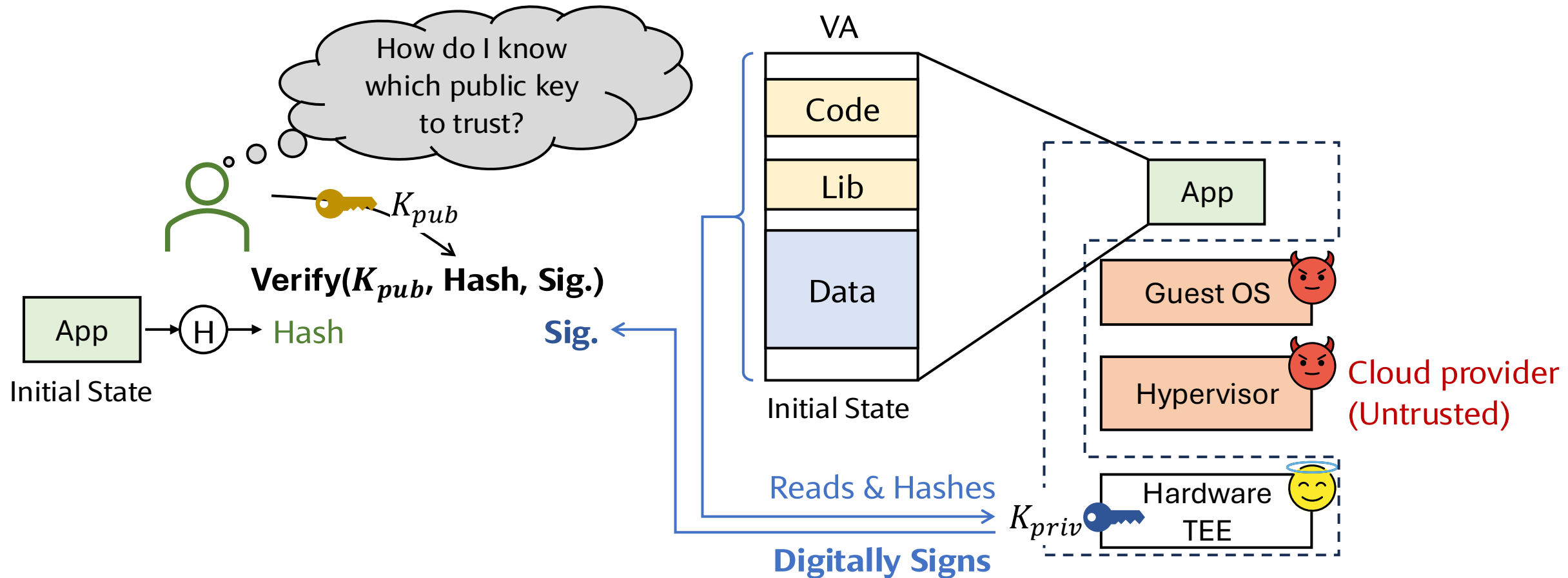
Note: Not all hash functions are cryptographic hash functions! E.g., Cyclic redundancy checks (CRCs) are not cryptographic hashes

Hammer 2: Digital Signature*

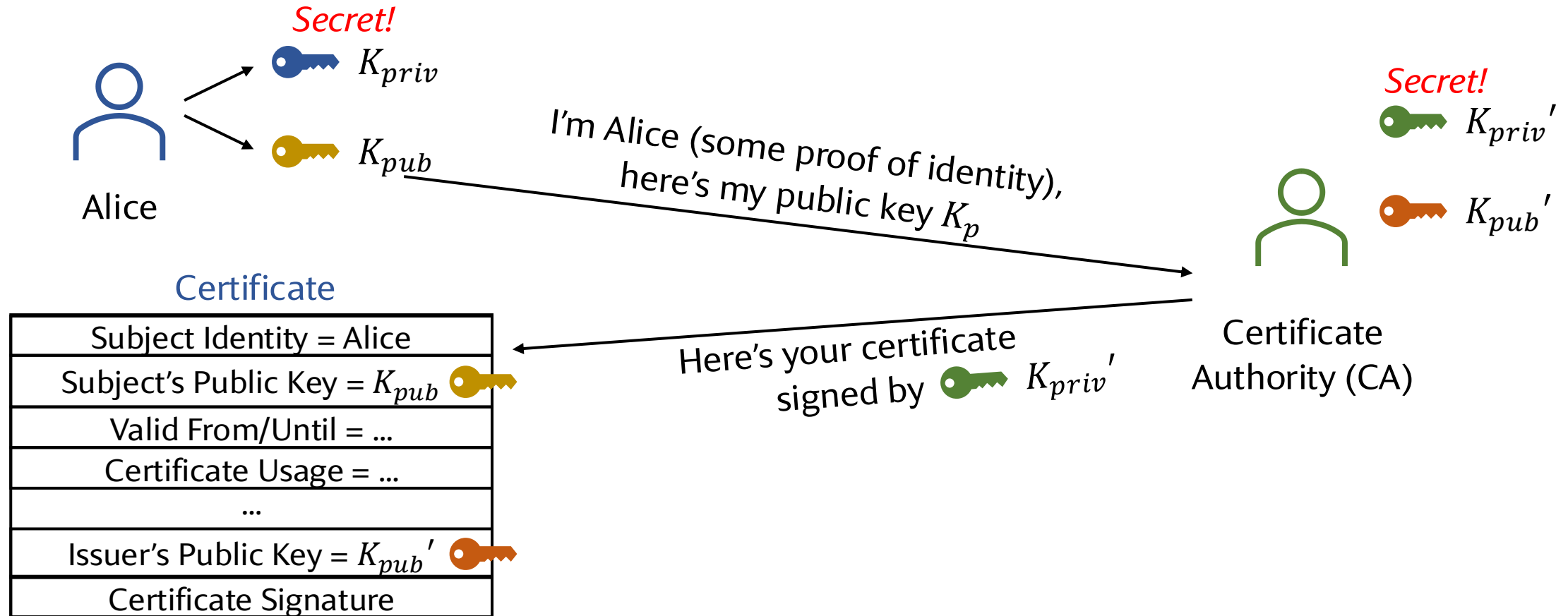


*Actual schemes are more complex than this

Software Attestation

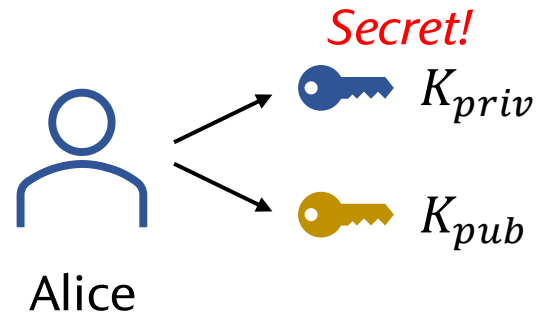


Public Key Infrastructure



A Certificate is Like an ID Card

It binds the subject's identity to their public key (or appearance)



Certificate

Subject Identity = Alice
Subject's Public Key = K_{pub}
Valid From/Until = ...
Certificate Usage = ...
...
Issuer's Public Key = K_{pub}'
Certificate Signature

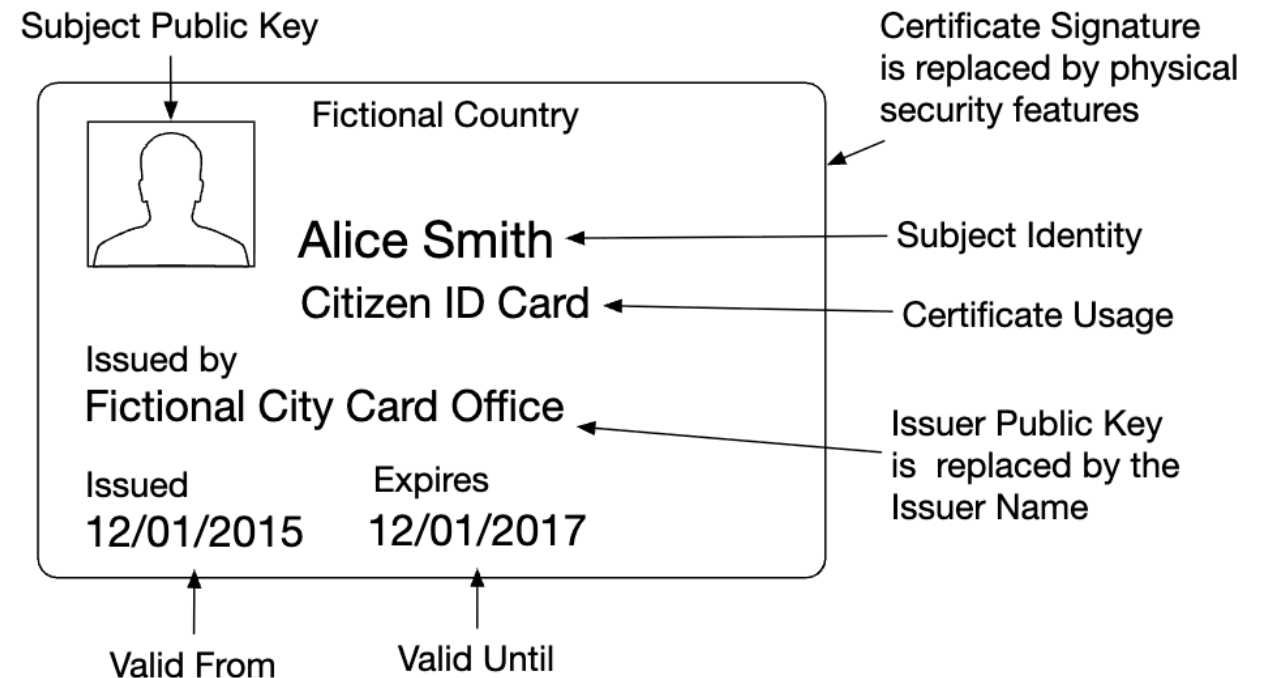
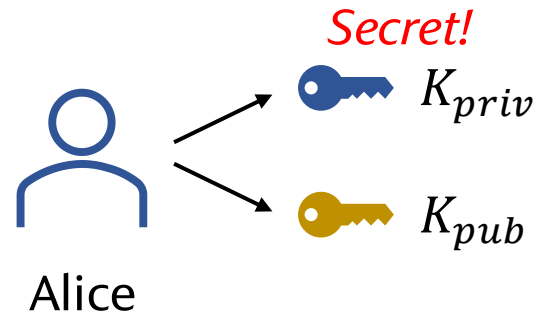


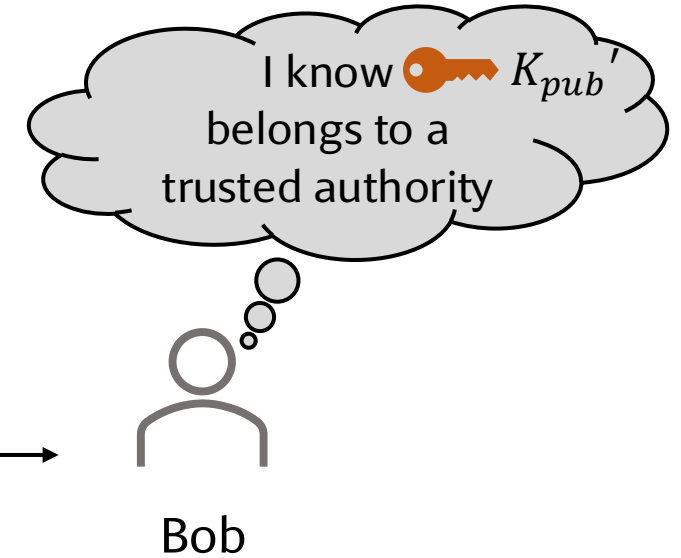
Illustration from "Intel SGX Explained" by Costan et al.

It's a proof of identity-pubkey binding, not proof of identity

Validating a Certificate



I'm Alice, here's my certificate.
It contains my public key



Certificate

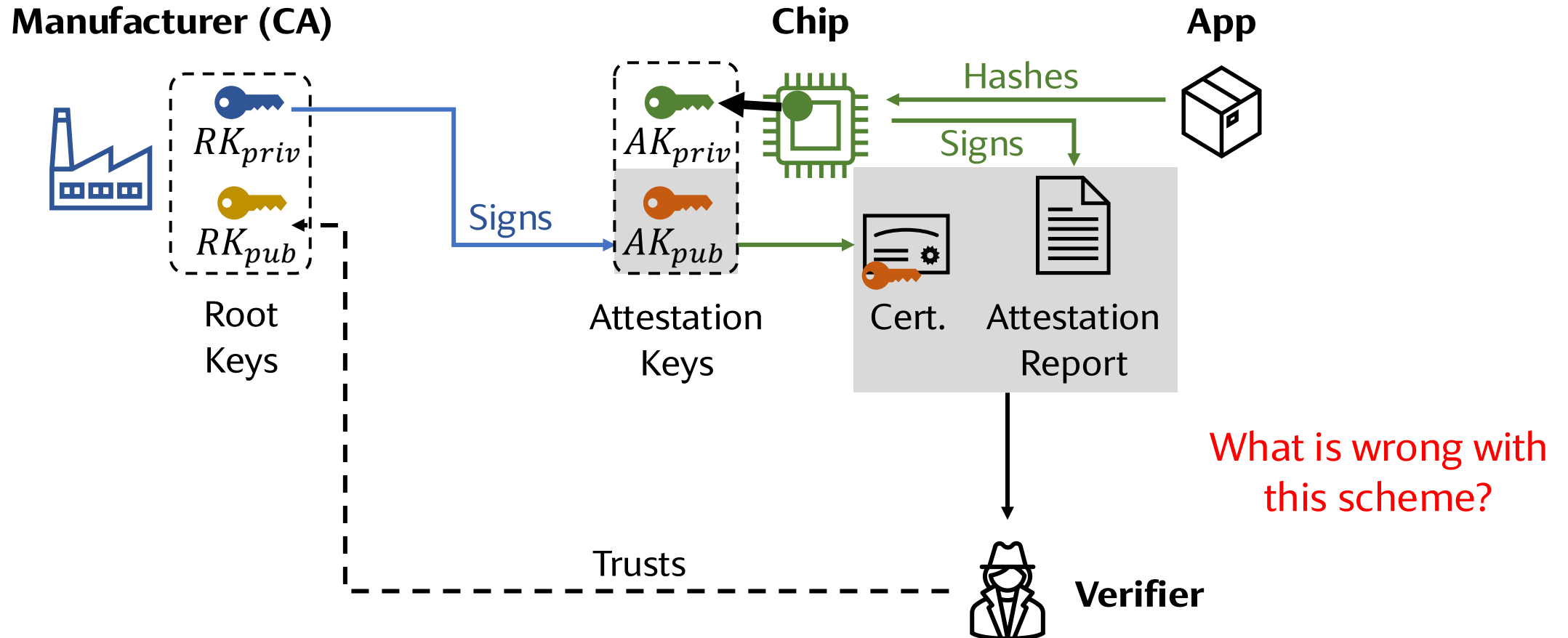
Subject Identity = Alice
Subject's Public Key = K_{pub}
Valid From/Until = ...
Certificate Usage = ...
...
Issuer's Public Key = K_{pub}'
Certificate Signature

Kicking the can
down the road?

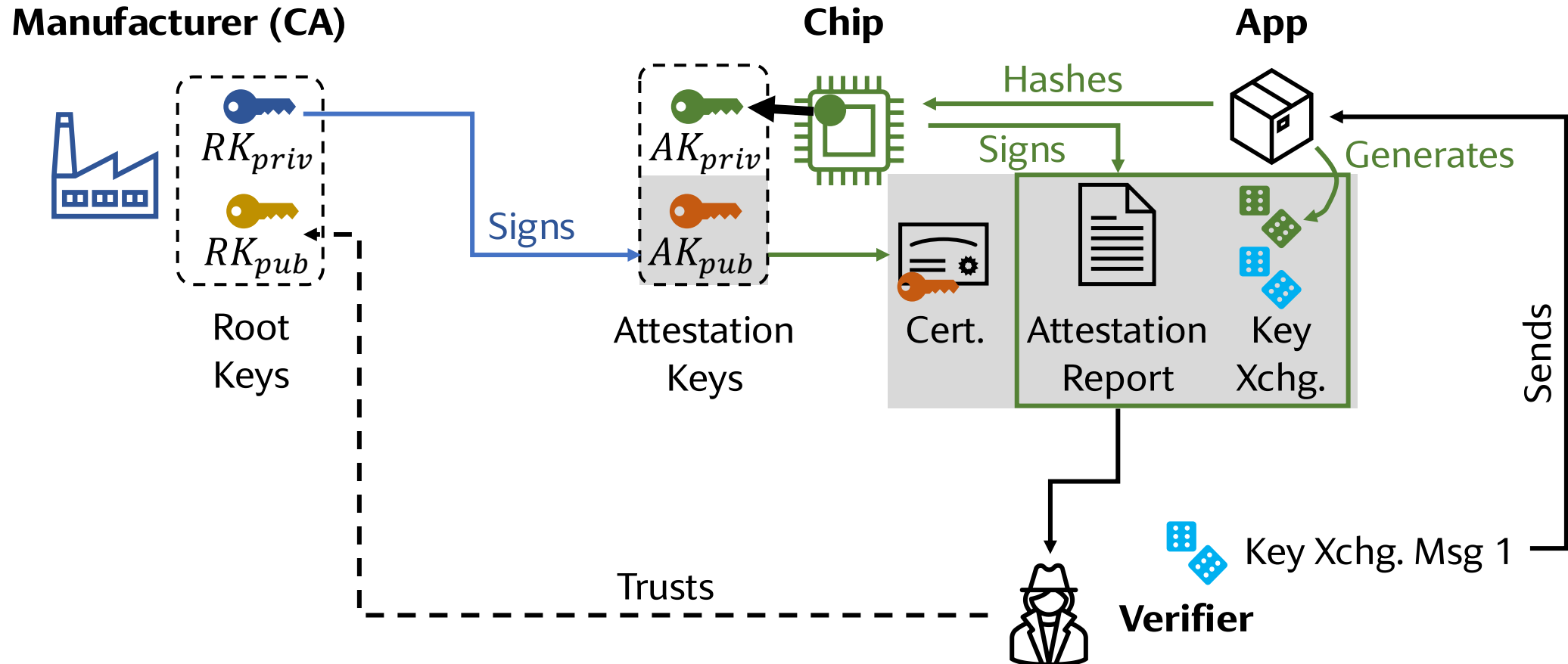
Validate the certificate:

- Does it say "Alice"?
- Expired?
- Valid signature?
- Does the issuer's public key belong to a trustworthy signer/CA
- ...

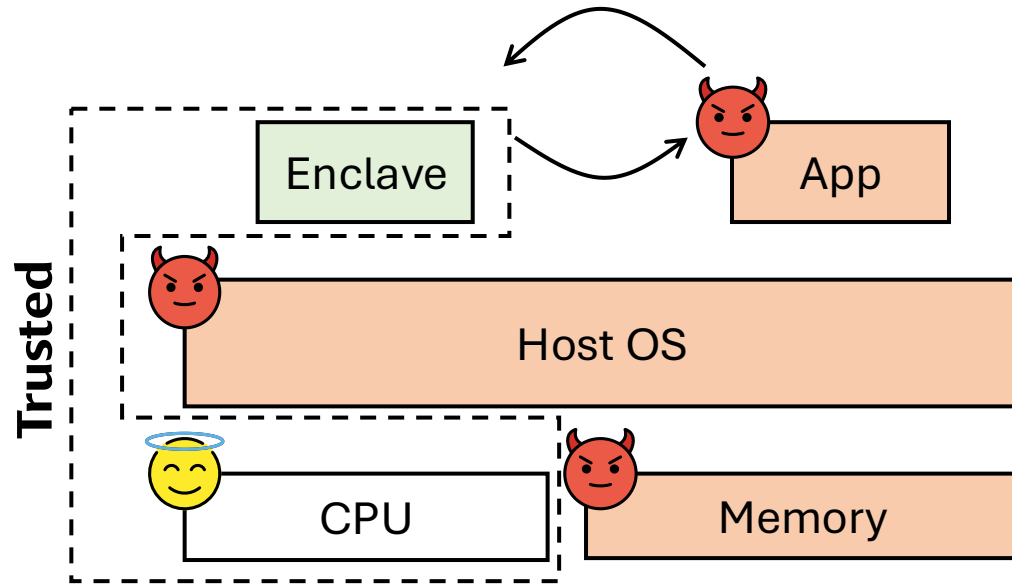
Putting All the Pieces Together



Putting All the Pieces Together



Intel SGX-1 Overview



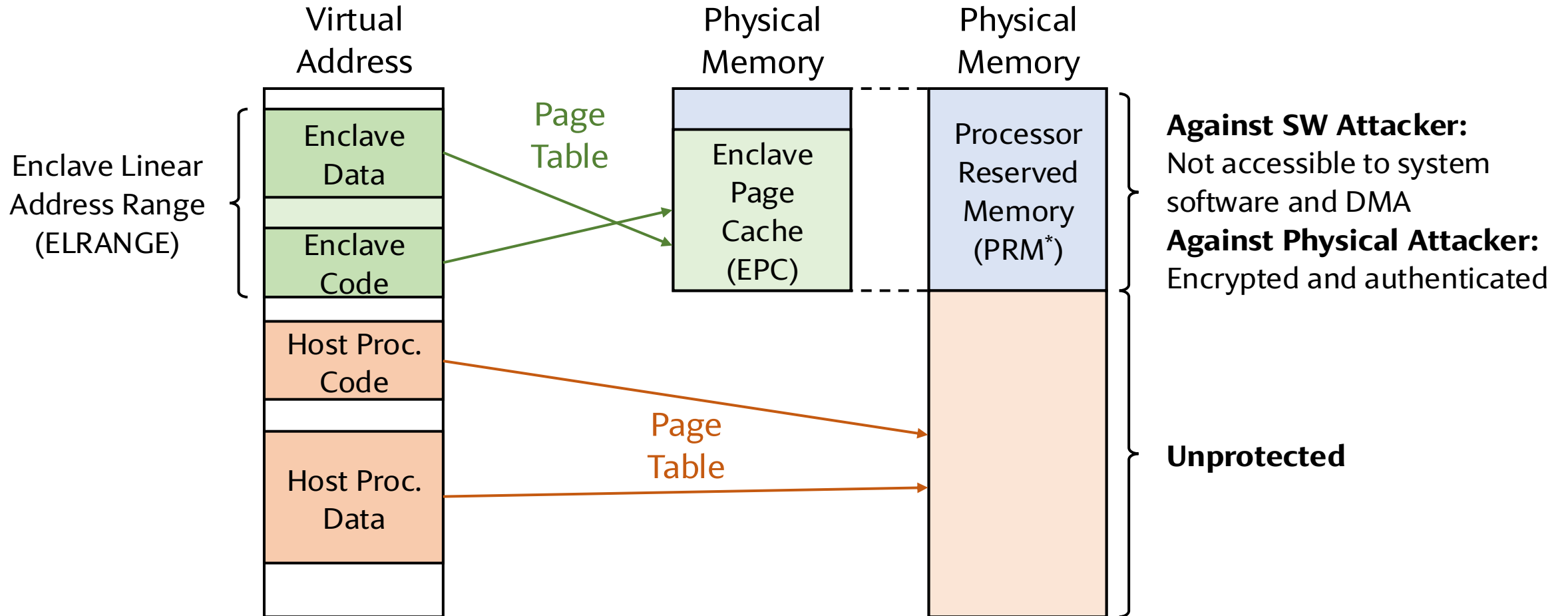
Resource management is delegated to the untrusted OS, who

- Allocates and frees memory
- Schedules enclave threads
- Serves interrupts
- ...

Before We Start

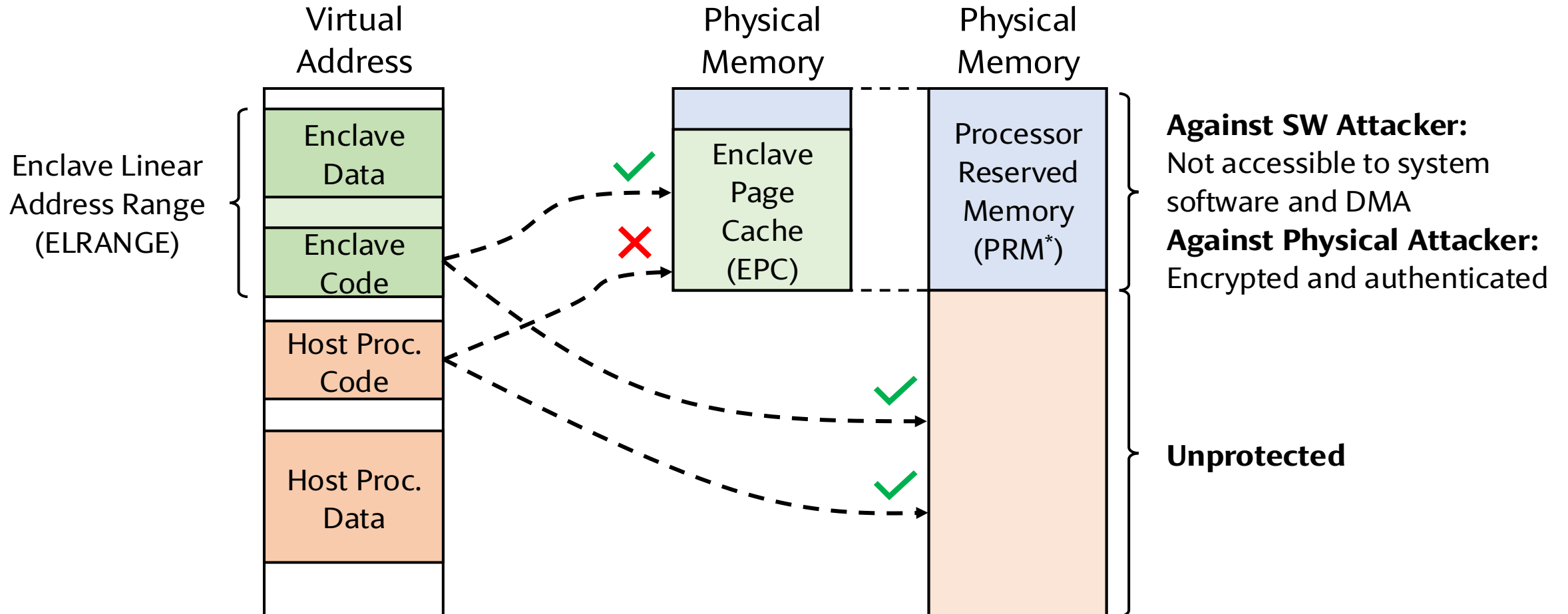
- We focus on the legacy Intel SGX-1
 - Successors: Intel SGX-2, Intel TDX
 - SGX-1 is well studied and serves as a good baseline to learn
- The exact Intel SGX design is complex, full of acronyms, and often undocumented. We simplified our discussion to help you understand the general TEE design challenges and solutions. Please consult Intel's Software Developer's Manual (SDM) on how to properly use it
- Why a certain design point is chosen is often undocumented. Therefore, some explanations are based on educated guesses

Intel SGX-1 Isolation Overview



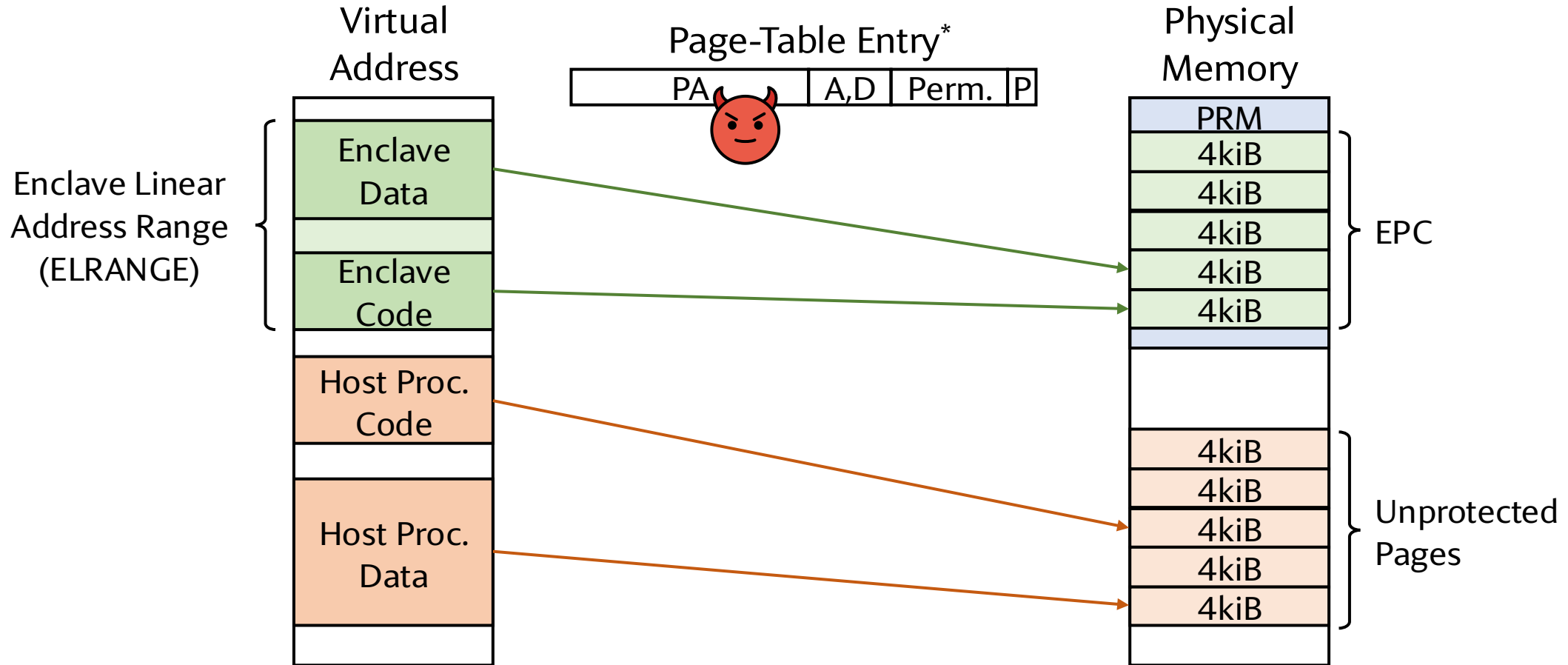
*Not to scale. PRM is often 128MB

Intel SGX-1 Isolation Overview



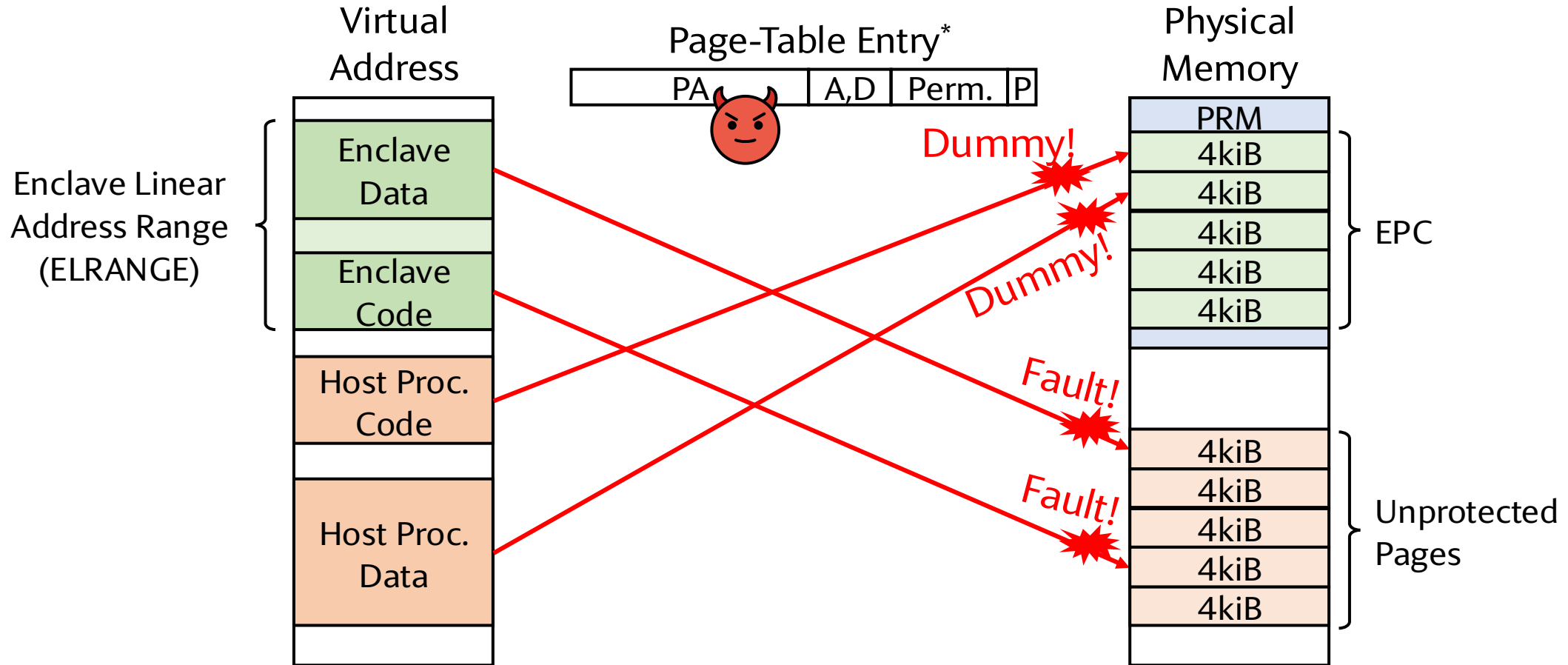
*Not to scale. PRM is often 128MB

The Untrusted OS Manages the Page Mapping



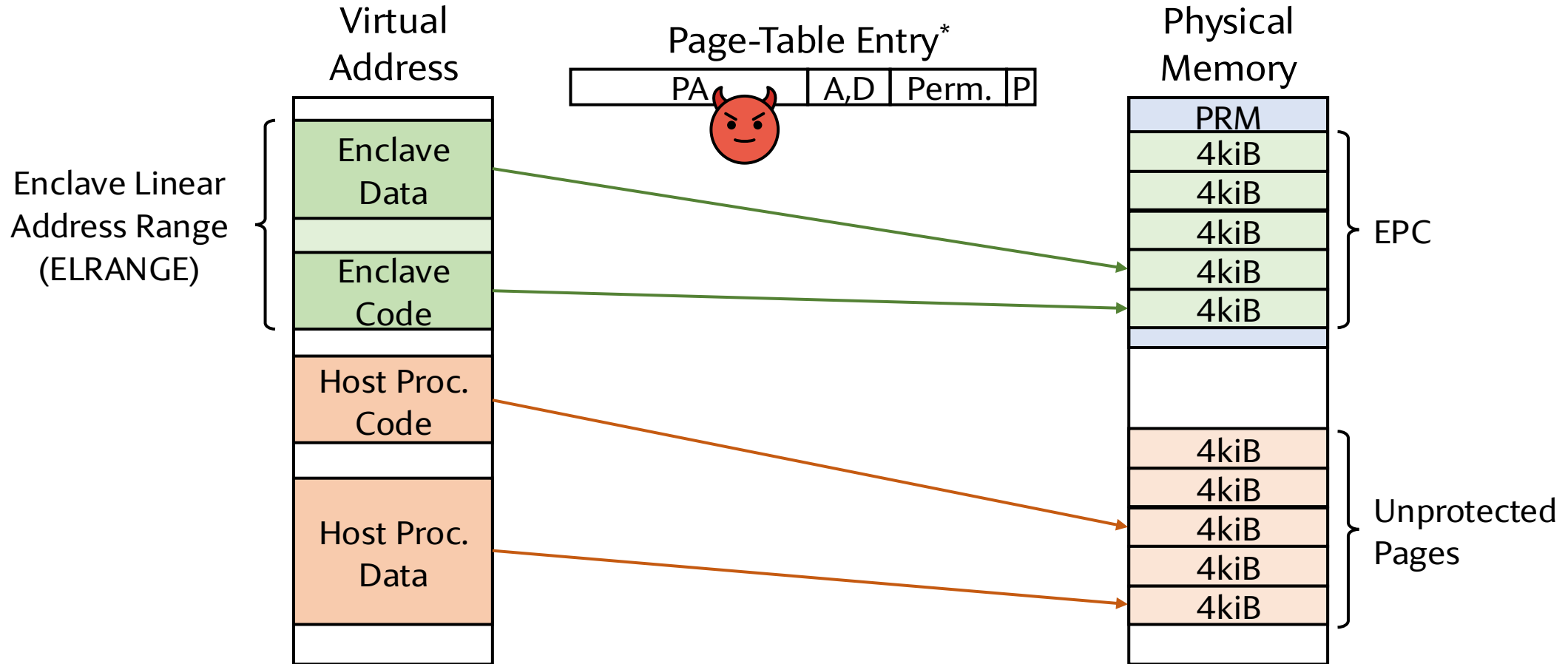
*Many fields omitted

The Untrusted OS Manages the Page Mapping

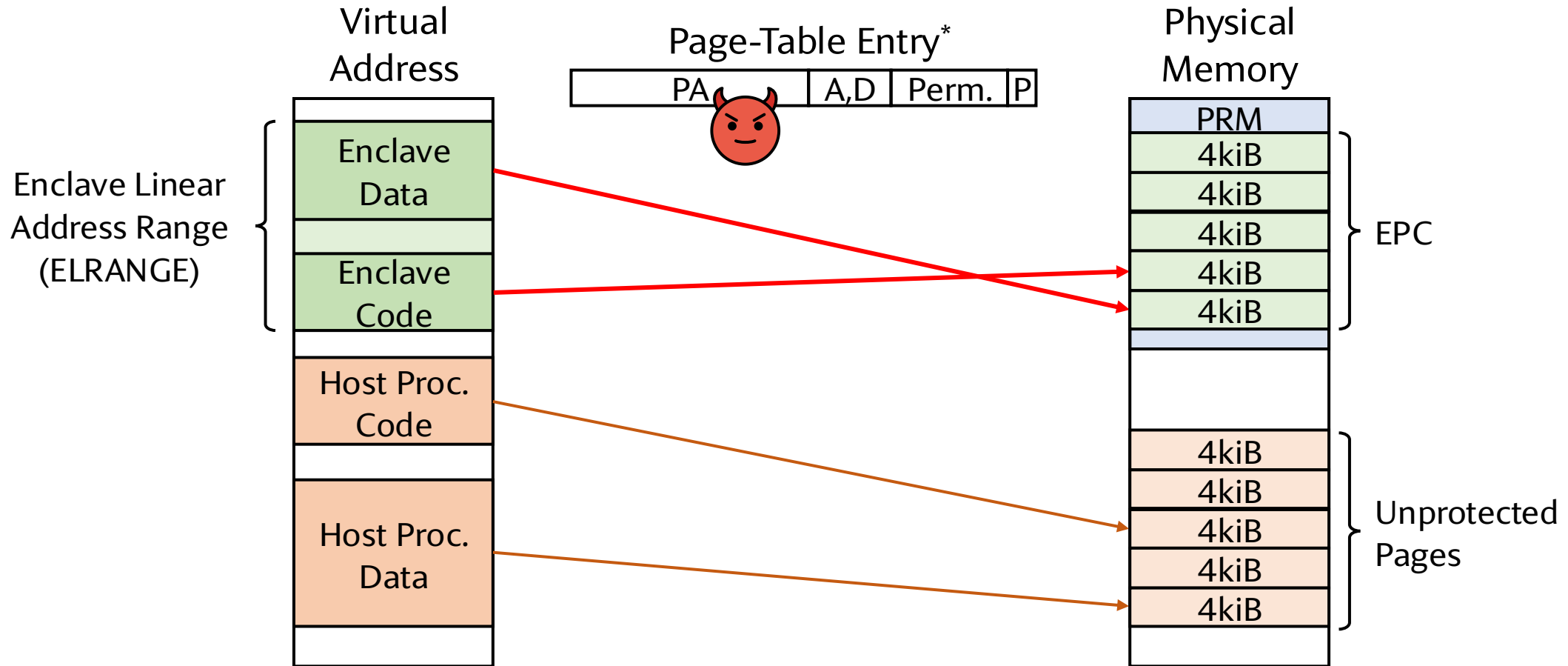


*Many fields omitted

What Else Can Go Wrong?

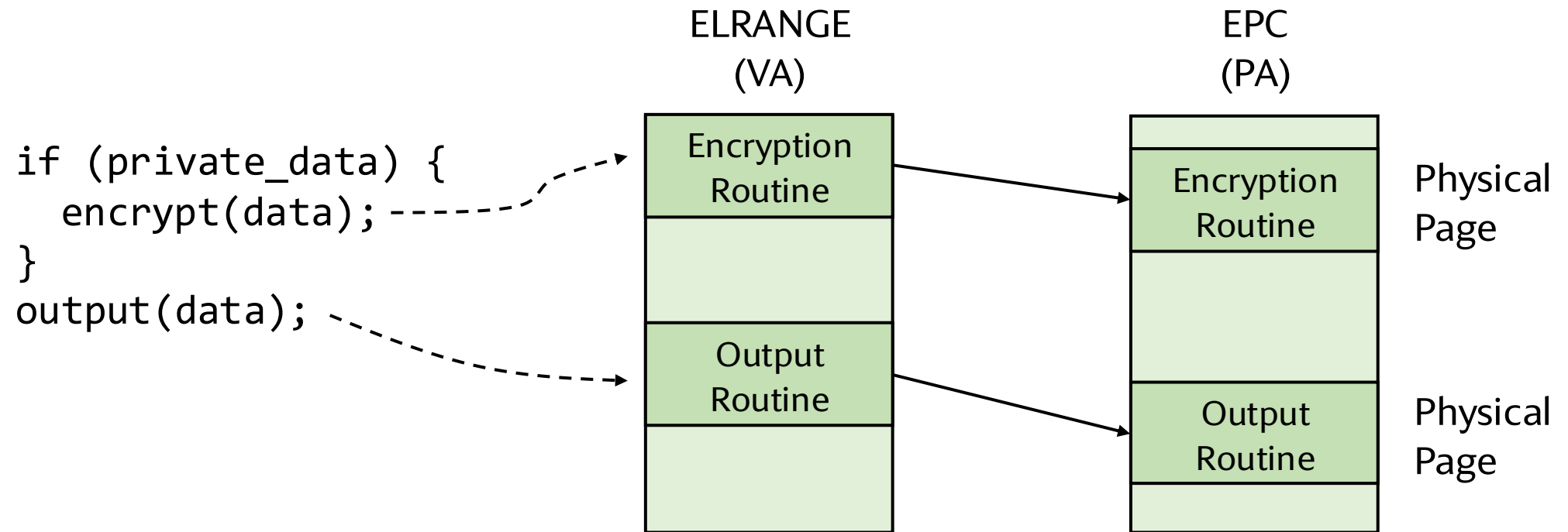


Page Swapping Attack



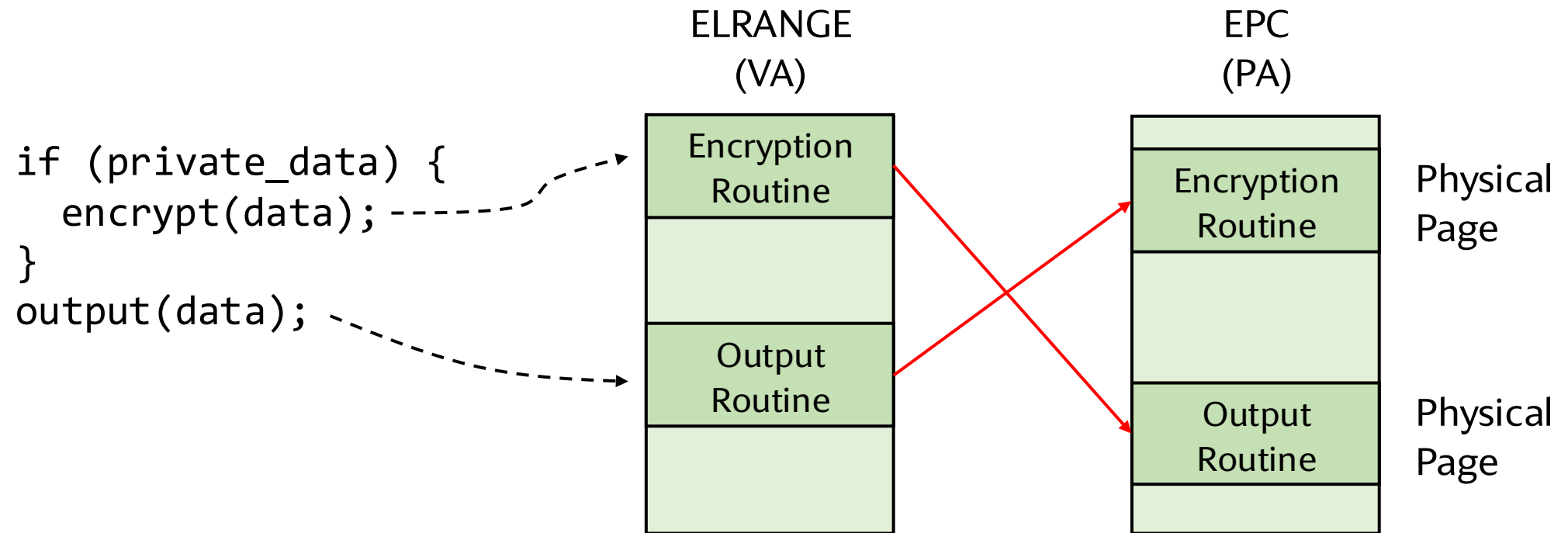
*Many fields omitted

Page Swapping Attack

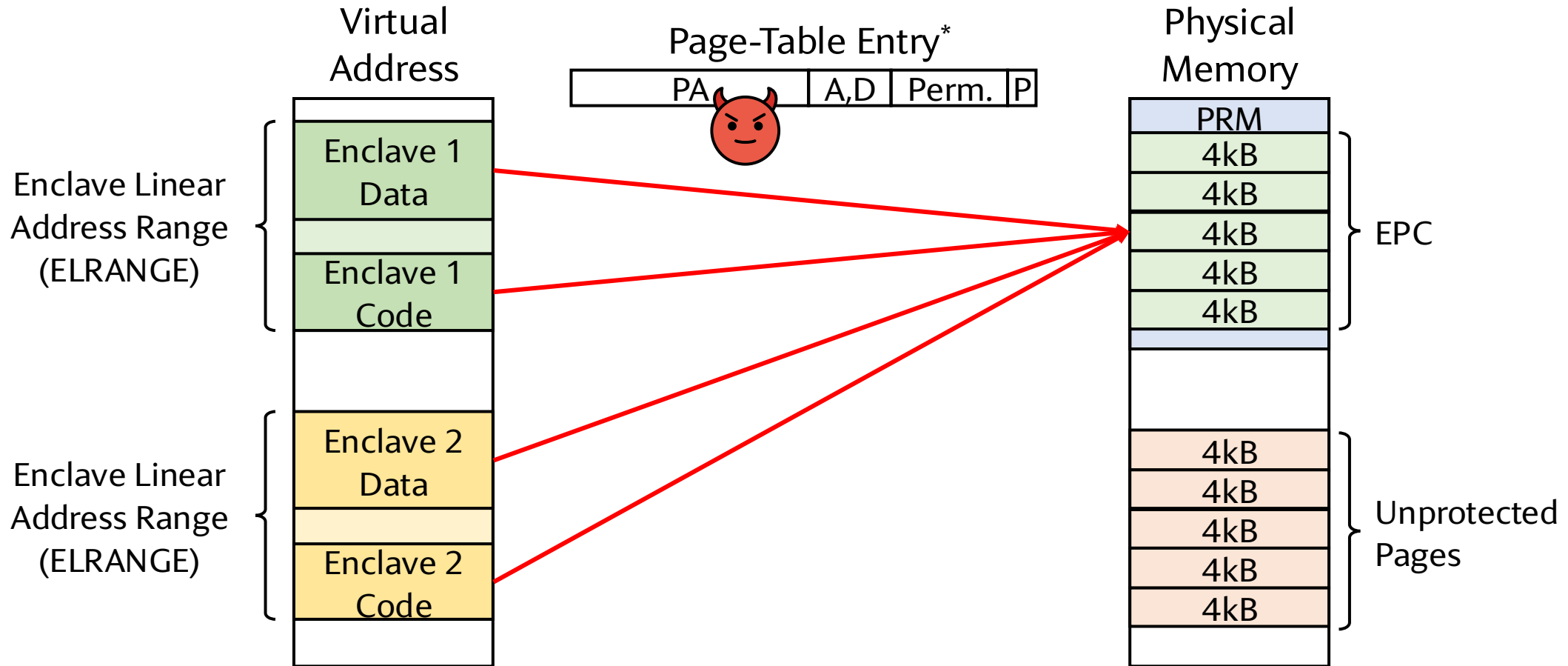


Page Swapping Attack

Private data are leaked without encryption

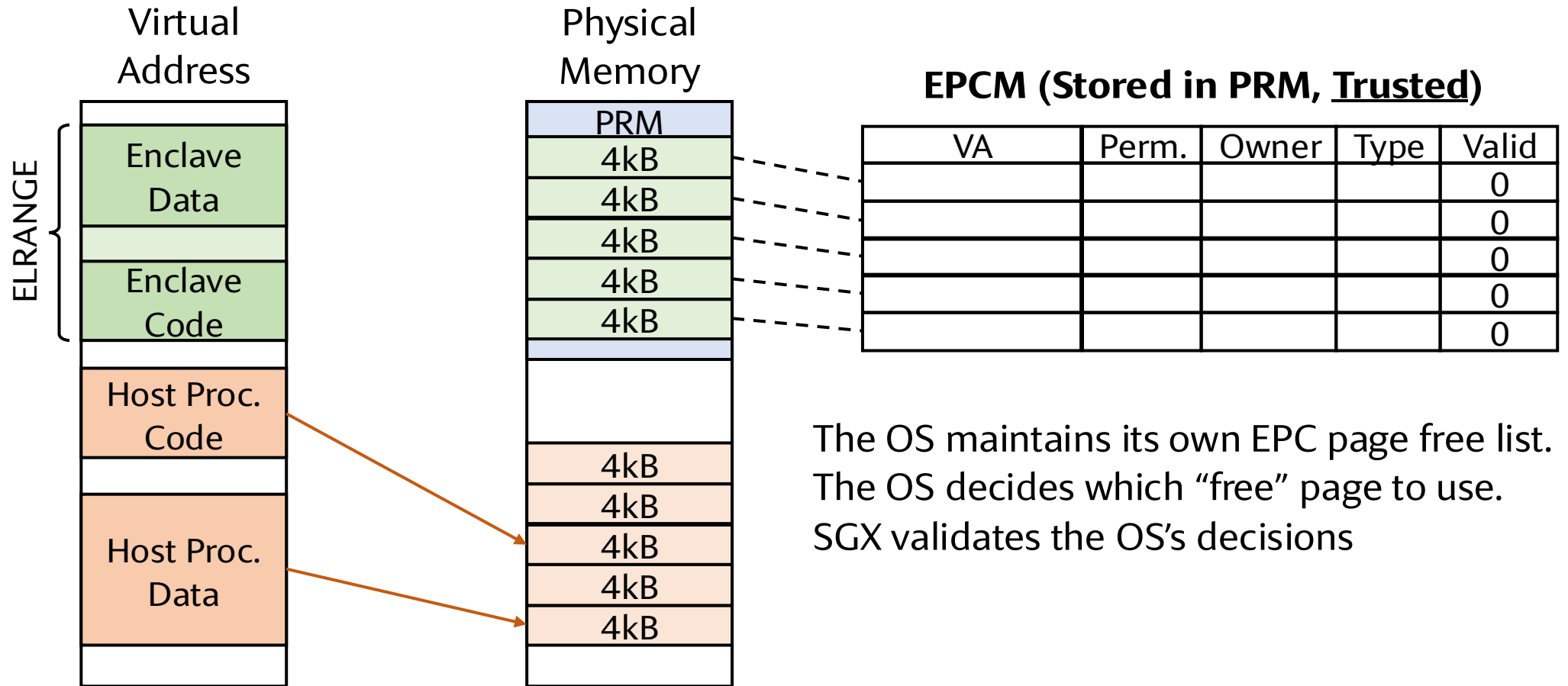


Intra- and Inter-Enclave Aliasing

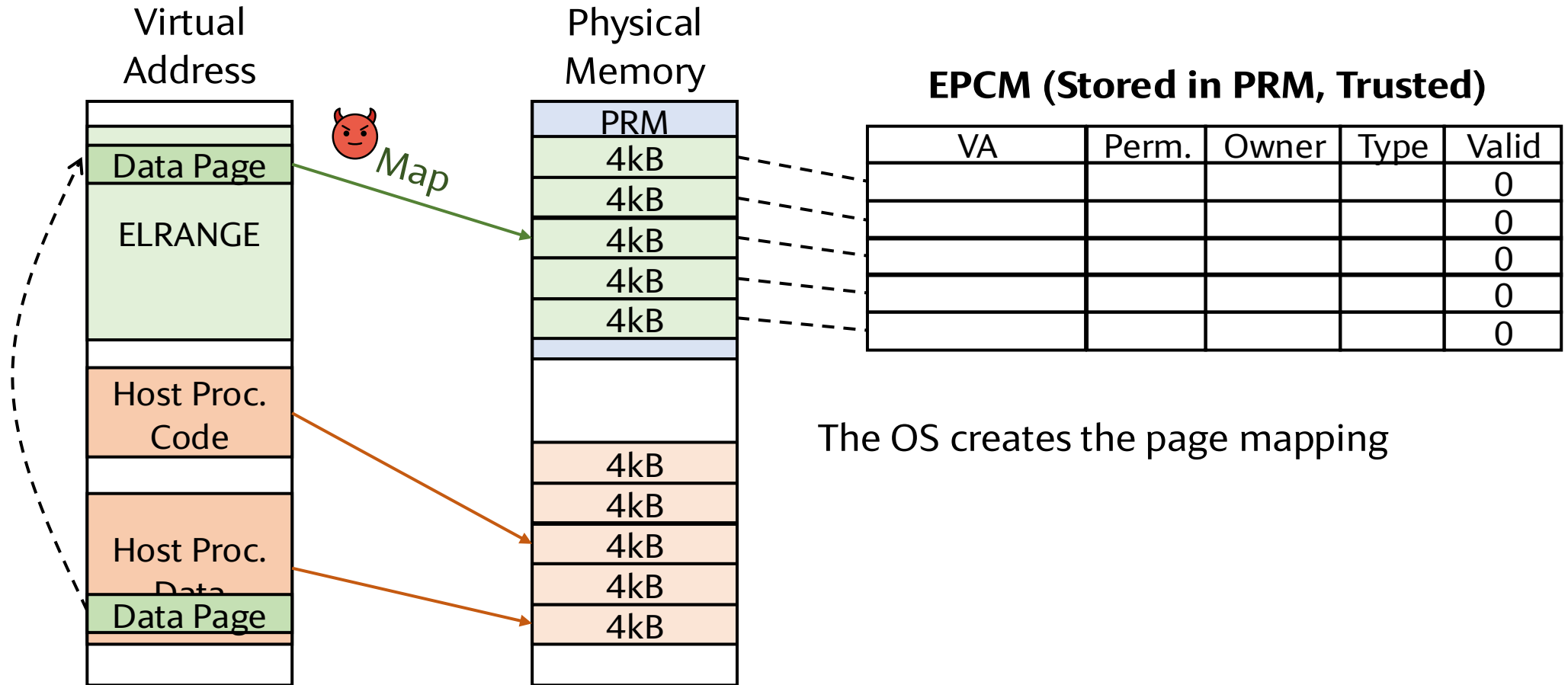


*Many fields omitted

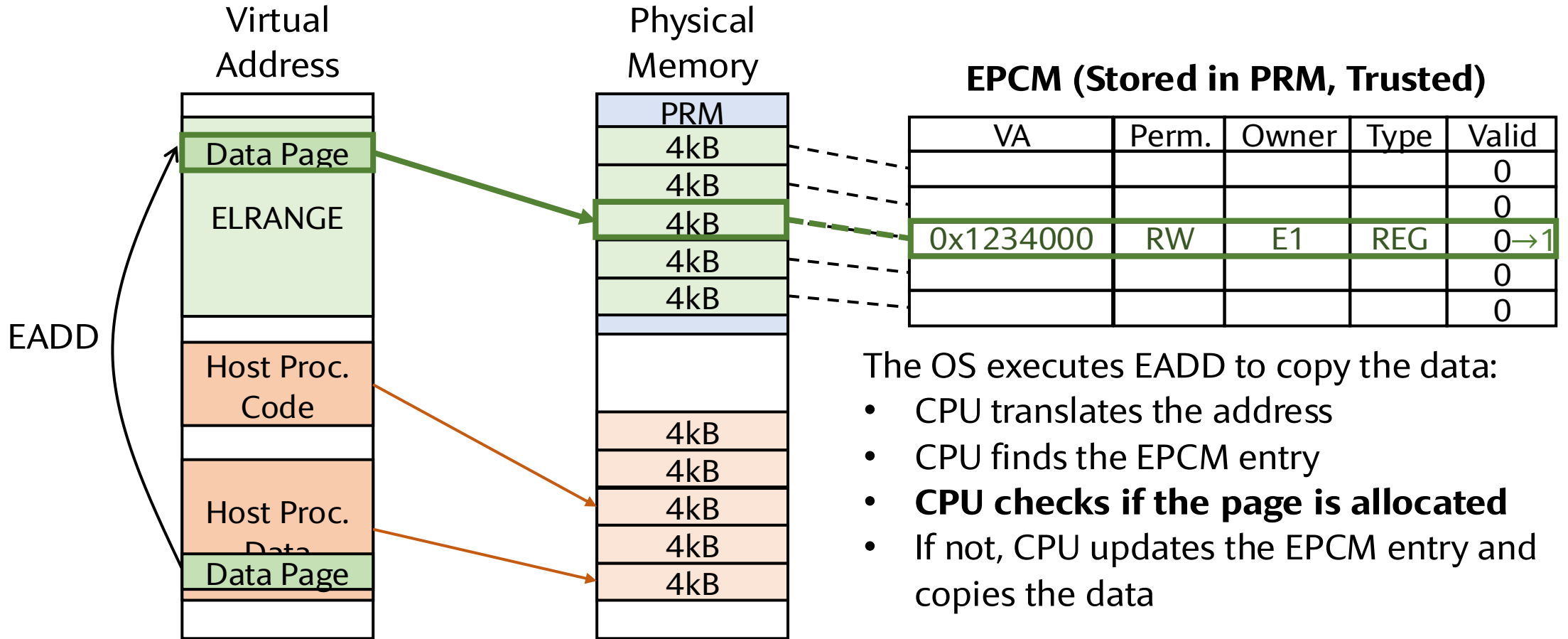
EPC Metadata (EPCM)



Allocating Memory and Enclave Initialization



Allocating Memory and Enclave Initialization

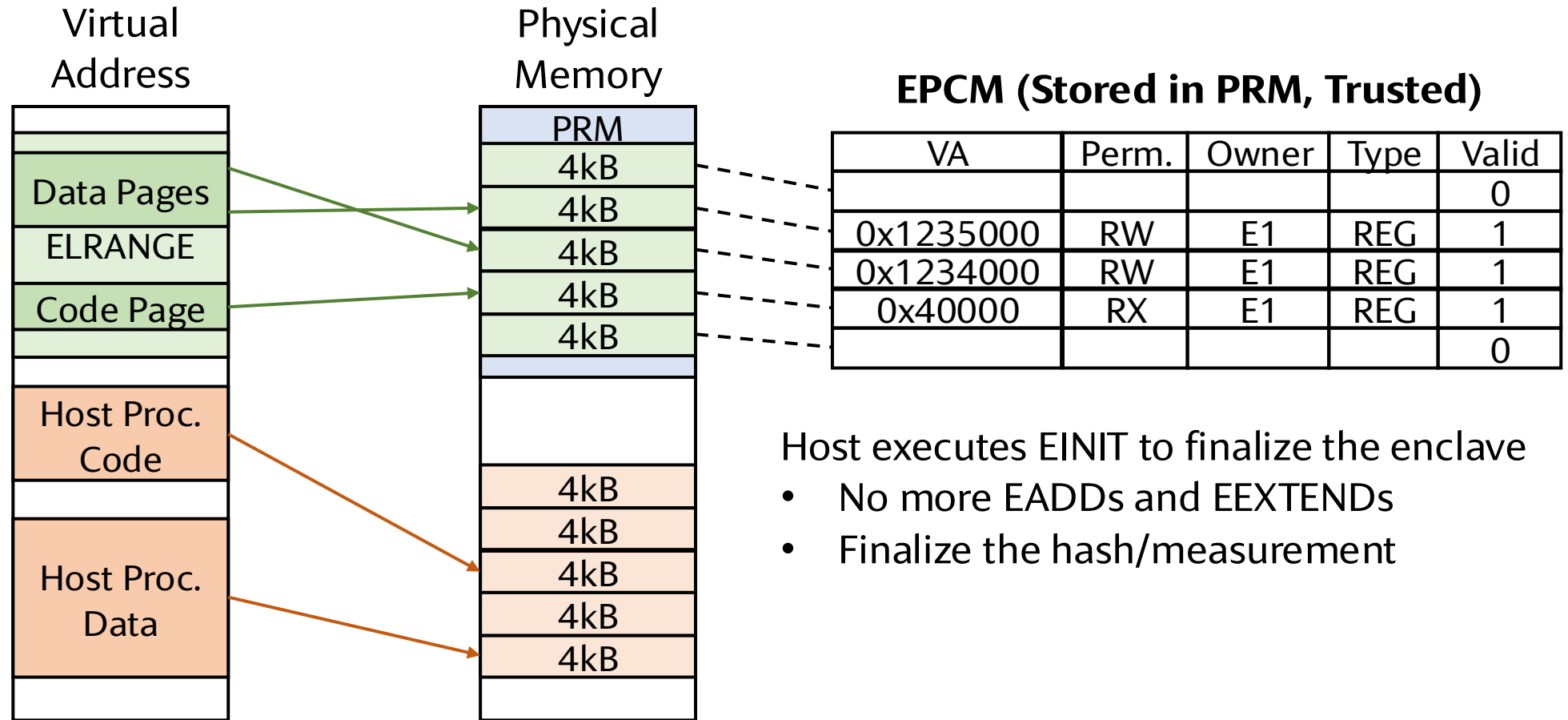


The OS executes EADD to copy the data:

- CPU translates the address
- CPU finds the EPCM entry
- **CPU checks if the page is allocated**
- If not, CPU updates the EPCM entry and copies the data

Host executes EEXTEND to hash the content

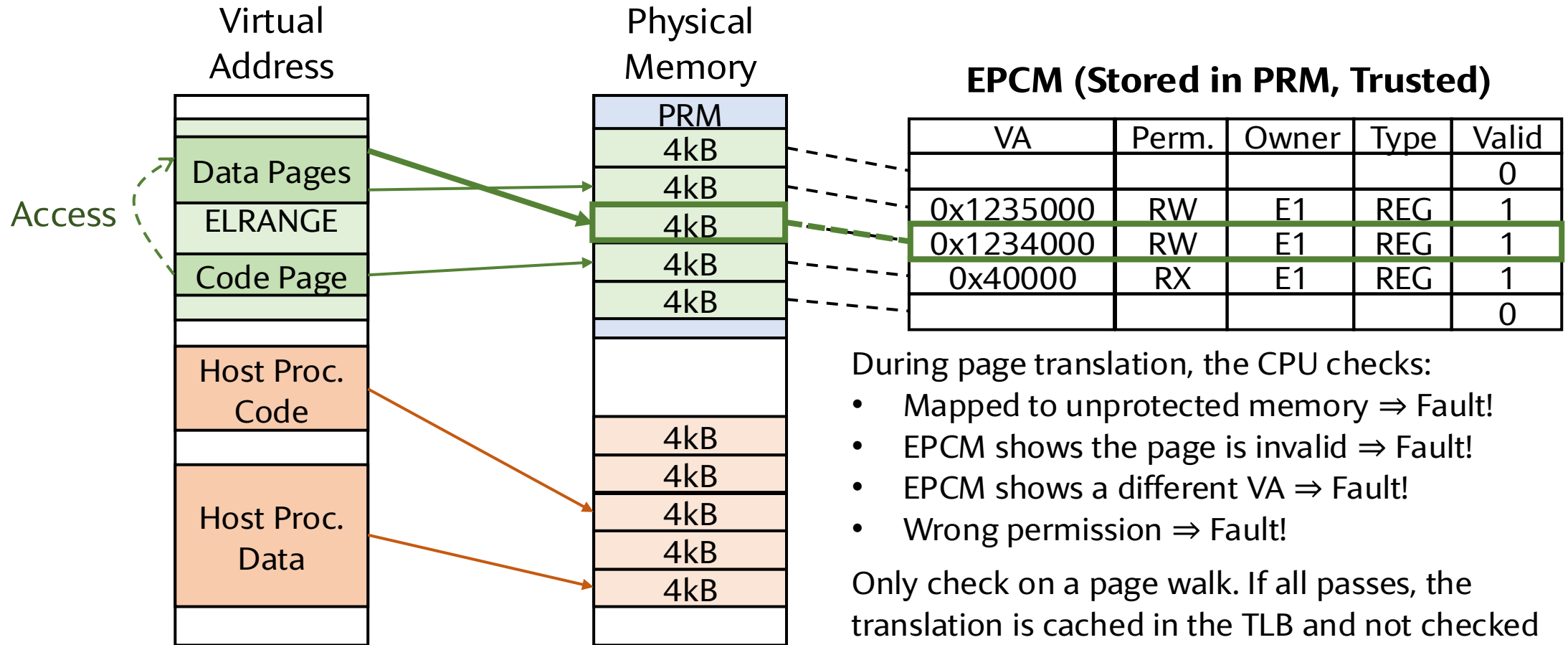
Allocating Memory and Enclave Initialization



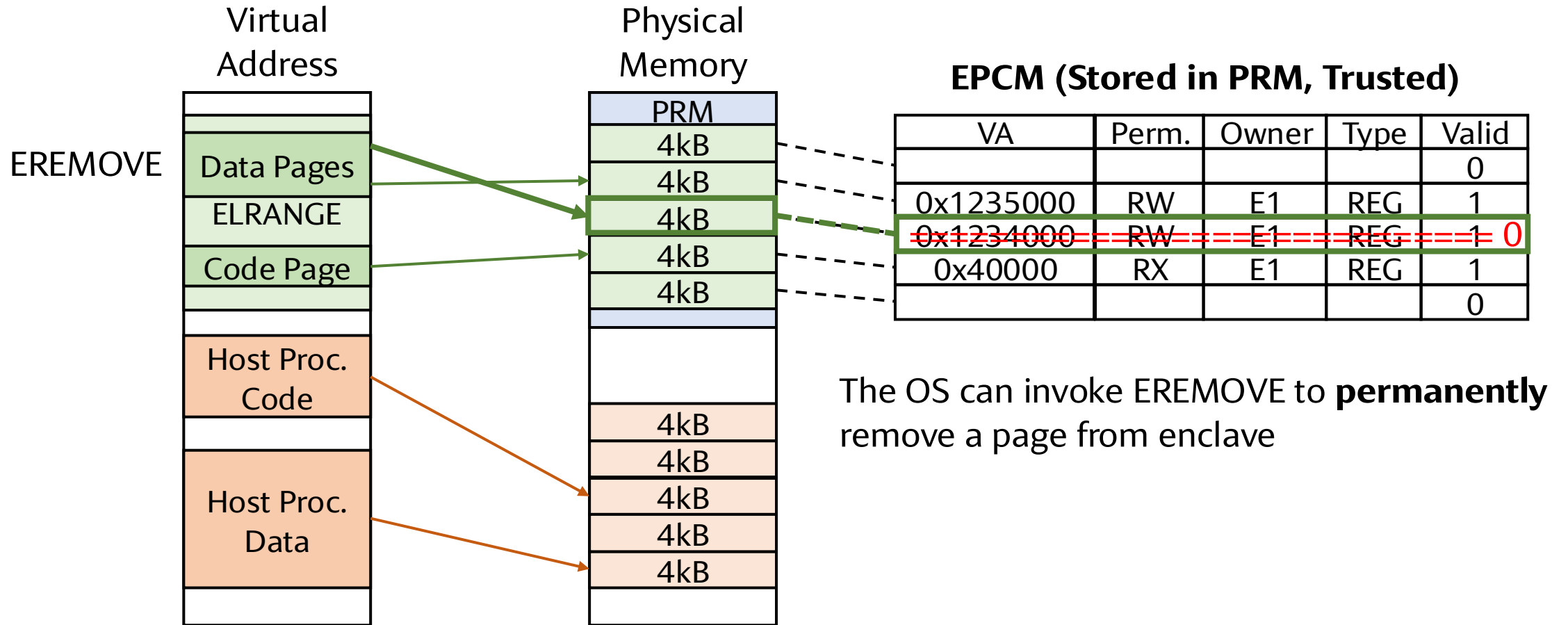
Host executes EINIT to finalize the enclave

- No more EADDs and EEXTENDs
- Finalize the hash/measurement

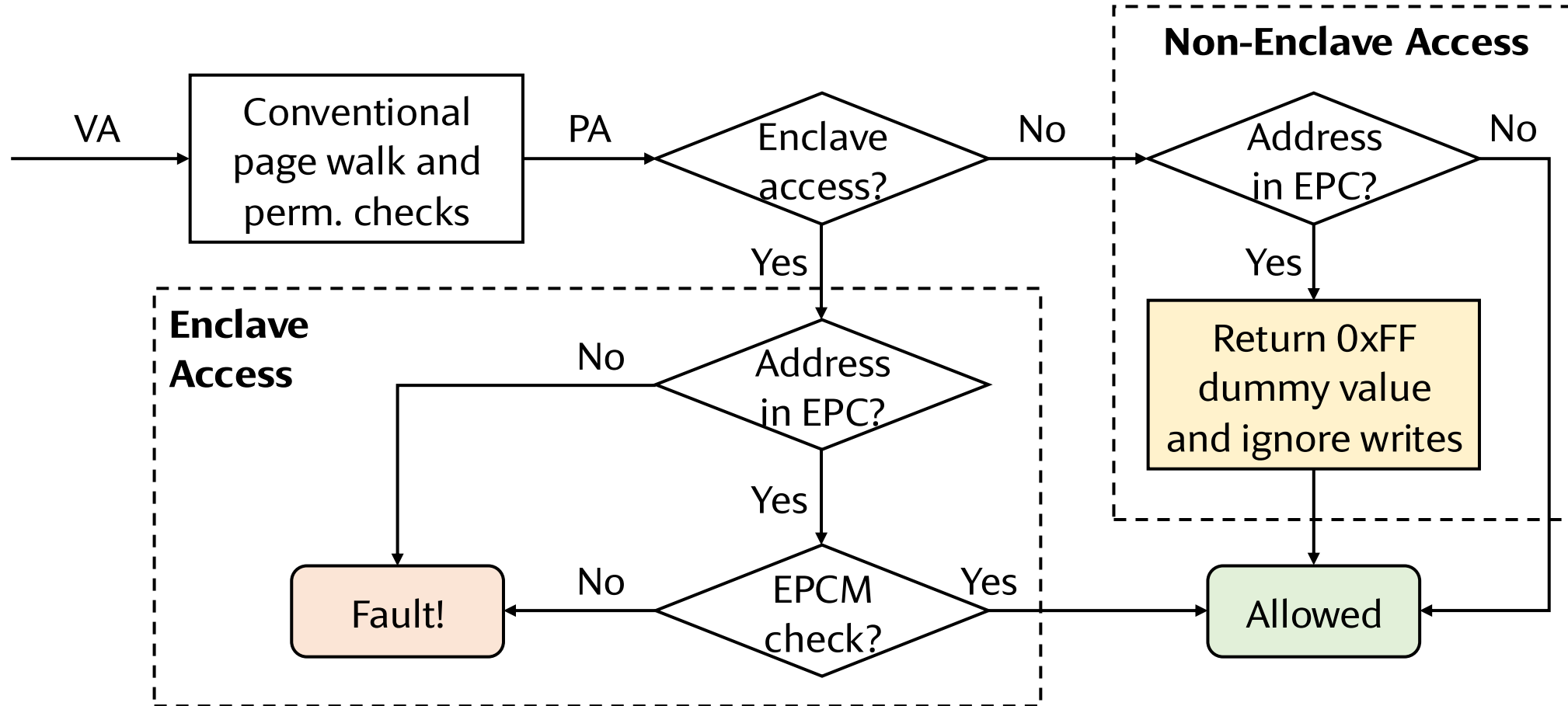
Runtime Checks Enforced by EPCM



Tearing Down an Enclave



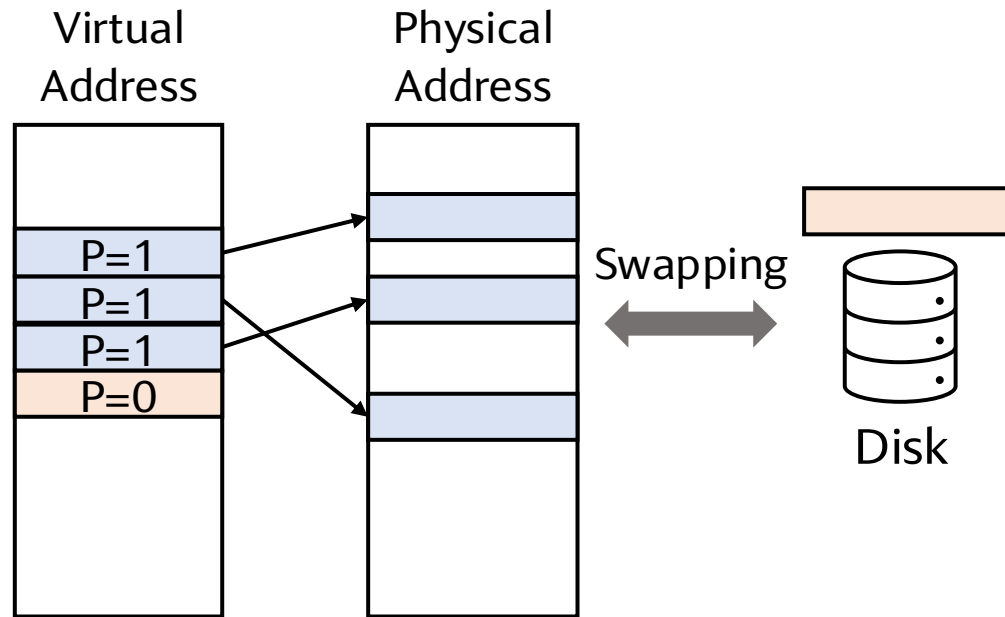
SGX-1 Access Control



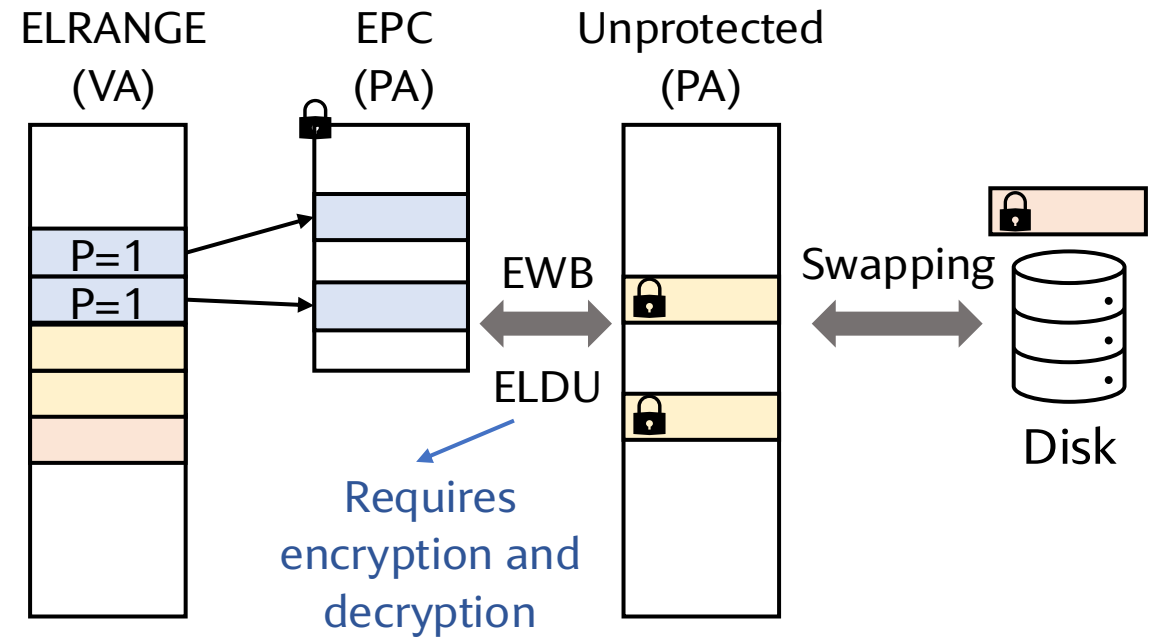
EPC Page Eviction

The maximum size of PRM is 128MB (SGX-1)

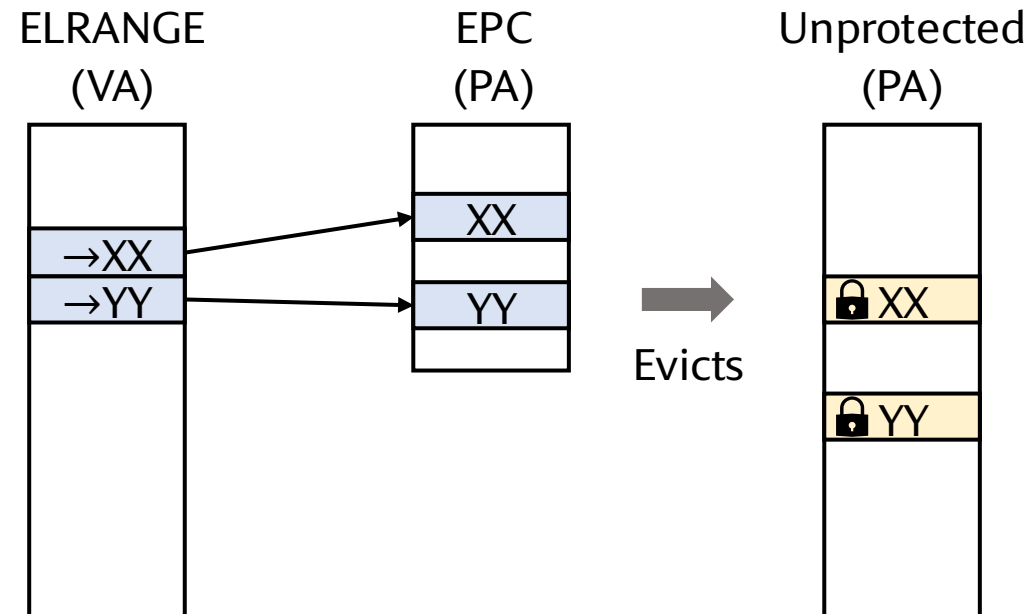
Traditional Paging



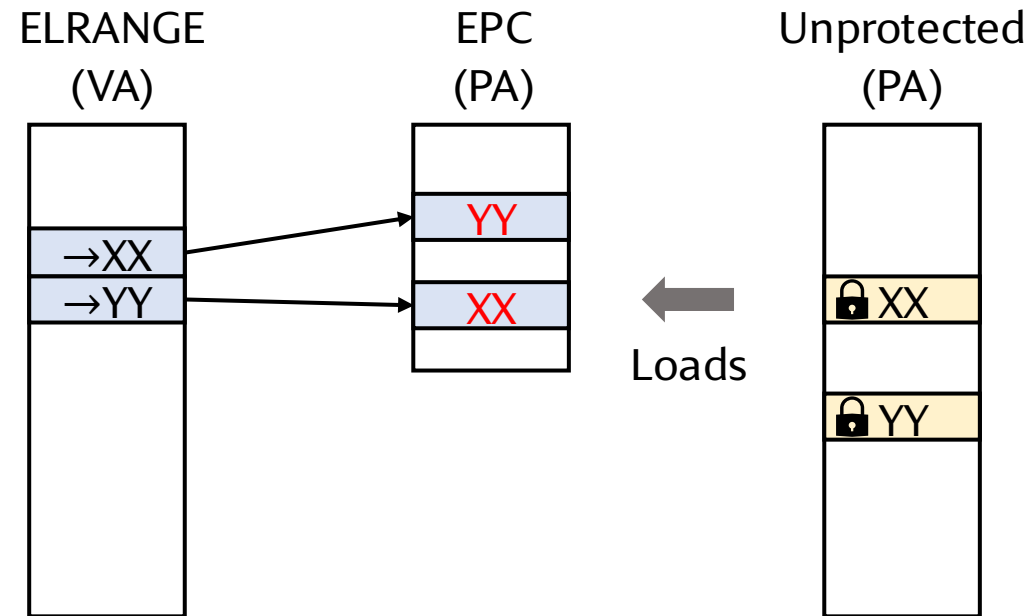
EPC Paging



Switcheroo (or Page Swapping) Keeps Trying to Sneak Back!



Switcheroo (or Page Swapping) Keeps Trying to Sneak Back!

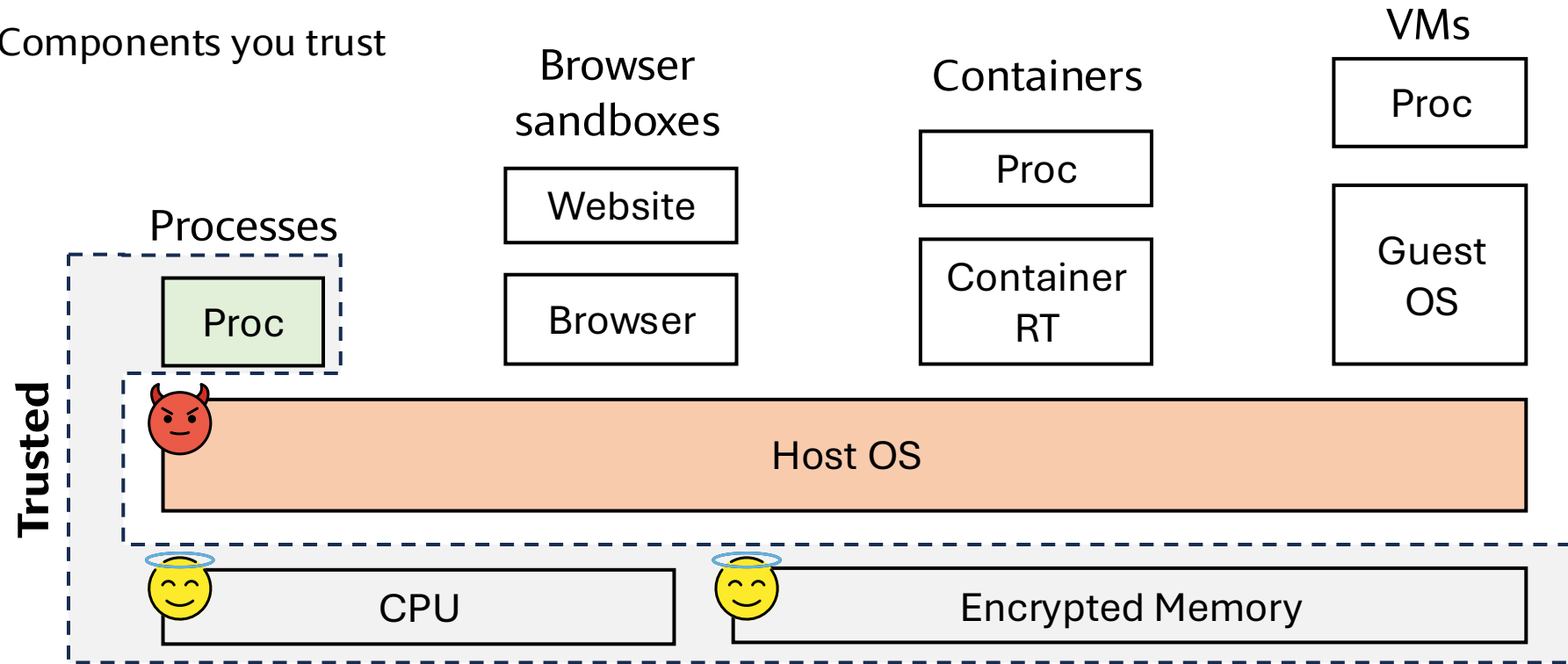


Intel SGX uses cryptographic mechanisms to prevent this

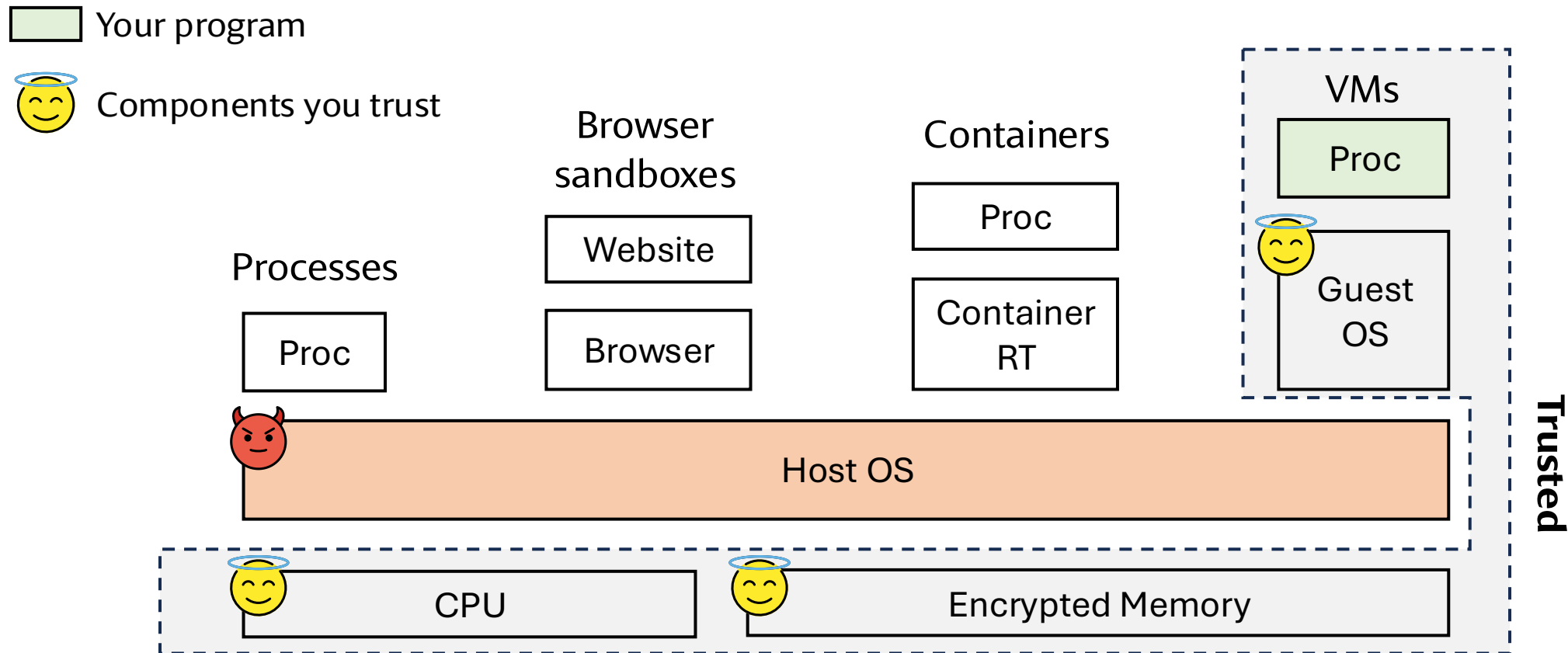
Intel SGX-1's Protection Model

 Your program

 Components you trust



Confidential Virtual Machine (CVM)



Examples: Intel TDX, AMD SEV-SNP

NVIDIA GPU Confidential Compute

